

**/\* This note has been compiled from various sources and can be taken as reference only \*/**

## **Chapter 6: OS Security and System Administration [3Hrs]**

### **I. OS Security Principles**

Operating system (OS) security refers to protecting the system from threats like unauthorized access, malware, and data breaches. It involves using techniques and mechanisms that ensure **confidentiality, integrity, and availability (CIA-Triad)** of system resources.

#### **1.1 Cryptography**

Cryptography is the science of securing data by converting it into unreadable form (encryption) and converting it back to readable form (decryption).

##### **Key Concepts:**

- **Encryption:** Converts plaintext into ciphertext using an algorithm and a key.
- **Decryption:** Reverses the process to recover the original data.

##### **Types of Cryptography:**

- **Symmetric Key Cryptography:**
  - Same key is used for encryption and decryption.
  - Example: **AES(Advanced Encryption Standard), DES (Data Encryption Standard)**
- **Asymmetric Key Cryptography:**
  - Uses a pair of keys: Public key (shared), Private key (kept secret).
  - Example: **RSA(Rivest-Shamir-Adleman), ECC(Elliptical Curve Cryptography)**

##### **Use in OS:**

- Secure file storage
- Data transmission between processes or systems
- Password storage (e.g., hashing passwords)

## 1.2. Multi-factor Authentication (MFA)- Imp

MFA is a login process that requires users to present two or more independent credentials.

### Common Authentication Factors:

1. **Something you know:** Password or PIN
2. **Something you have:** Mobile device, smart card, OTP token
3. **Something you are:** Fingerprint, retina scan, voice recognition

**2FA utilizes combination of at least two of above three groups (1+2 or 1+3 or 2+3)**

### Use in OS:

- Secure login to the system
- Enhances user verification in remote access (e.g., SSH with OTP)

## 1.3. Secure Booting

Secure booting is a security standard that ensures only trusted and verified software is loaded during the startup process of a system.

### How it works:

- Uses **digital signatures** to verify the bootloader and OS kernel.
- Prevents **rootkits, boot sector viruses, and unauthorized OS modifications.**

### Use in OS:

- Found in **UEFI(Unified Extensible Firmware Interface)** firmware in modern systems.
- Common in mobile OS (e.g., Android Verified Boot).

## 1.4. Sandboxing- IMP

Sandboxing is a security technique where applications are run in a **restricted environment**, isolated from the rest of the system.

## Purpose:

- Prevents an app from accessing system files, other apps, or user data.
- Limits the impact of malware or buggy programs.

## Use in OS:

- Web browsers (e.g., Chrome tabs are sandboxed)
- Mobile OS (e.g., Android and iOS sandbox each app)
- Running untrusted programs safely in Linux using tools like **Firejail**, **Docker**, or **Flatpak**

## Summary Table:

| Technique                   | Purpose                             | Example Use Case                         |
|-----------------------------|-------------------------------------|------------------------------------------|
| Cryptography                | Secure data via encryption          | Password storage, file protection        |
| Multi-factor Authentication | Verify user with more than 1 factor | Logging into secure systems              |
| Secure Booting              | Protect boot process from tampering | <b>UEFI</b> firmware checks OS signature |
| Sandboxing                  | Isolate applications                | Safe browser tab or app execution        |

## Questions for Students:

1. What is the difference between symmetric and asymmetric encryption?
2. Why is MFA more secure than using only a password?
3. What does Secure Boot protect against?
4. Give one real-world example of sandboxing.

## 2. Access Control Mechanisms in Operating Systems

### Introduction

Access control in an operating system refers to the set of techniques used to **regulate which users or processes can access specific system resources** (such as files, devices, or memory). It ensures **confidentiality, integrity, and proper use of resources**.

Access control mechanisms typically involve **policies, Access Control Lists (ACLs), capabilities, and specific OS support structures**.

#### 2.1. Access Control Policies- Imp

Policies define the **rules and principles** that determine who can access what.

##### Types of Policies:

##### 1. Discretionary Access Control (DAC)

- Owner of the resource decides who has access.
- Example: File permissions in Linux (e.g., chmod).

##### 2. Mandatory Access Control (MAC)

- Access is controlled by a central authority based on classification levels.
- Common in military systems.
- Example: SELinux, AppArmor.

##### 3. Role-Based Access Control (RBAC)

- Access is granted based on roles (e.g., admin, user, guest).
- Simplifies large-scale system administration.

#### 2.2. Access Control Lists (ACLs) -Imp

ACL is a **list associated with an object (like a file or directory)** that defines which users or groups can access that object and how.

##### Structure:

- Each entry in an ACL specifies:
  - A **subject** (user or group)
  - A **set of permissions** (read, write, execute)

### Example (Linux ACL):- Explain

```
bash
getfacl file.txt
```

```
text
# file: file.txt
user::rw-
user:john:rw-
group::r--
other::---
```

This means the file is:

- Readable and writable by the owner
- Readable and writable by user John
- Readable by the group
- No access for others

### 2.3. Capabilities

Capabilities are an alternative to ACLs where **permissions are associated with the subject (user/process)**, not the object.

#### Concept:

- A capability is like a **token or key** that gives the holder specific rights over an object.
- The OS ensures that only processes holding the capability can perform operations.

#### Example Use:

- File descriptors in Unix/Linux systems (e.g., a process must have a valid file descriptor to read/write a file).
- Capability-based microkernels like **seL4**.

## Difference from ACLs:

| ACLs                            | Capabilities                     |
|---------------------------------|----------------------------------|
| Permissions stored with objects | Permissions stored with subjects |
| System checks list of users     | System checks user's token       |

## 2.4. OS Support for Access Control

Modern operating systems provide extensive support for implementing access control mechanisms.

### Examples:

- **UNIX/Linux:**
  - DAC using file ownership and permissions (chmod, chown)
  - ACLs via getfacl/setfacl
  - MAC via SELinux or AppArmor
- **Windows:**
  - NTFS file system with ACLs
  - Group Policy for RBAC
- **Android:**
  - Uses Linux kernel + SELinux for MAC
  - Application sandboxing based on UID

### Summary Table:

| Mechanism         | Description                       | Example                |
|-------------------|-----------------------------------|------------------------|
| Policy (DAC, MAC) | Rules for granting/denying access | Linux chmod, SELinux   |
| ACL               | List of permissions per object    | NTFS, Linux ACLs       |
| Capabilities      | Rights assigned to subjects       | File descriptors, seL4 |
| OS Support        | Tools and APIs provided by the OS | chmod, Group Policy    |

**Quick Questions for Students:**

1. What is the difference between DAC and MAC?
2. How do ACLs and capabilities differ in access control?
3. How does Linux implement access control?
4. Give an example of OS support for Role-Based Access Control.

## 3. User Account Management and Permissions in Operating Systems

### Introduction

User account management and permissions are fundamental for controlling access to system resources and maintaining security. User accounts identify individuals or processes, while permissions define what operations those accounts can perform.

### Objectives

- Authenticate and authorize users.
- Assign appropriate permissions for resource access.
- Isolate user environments and data.
- Monitor user activities for security.
- Manage user lifecycle effectively.

### 3.1. User Account Components

A user account typically includes:

- **Username:** Unique identifier for the user.
- **User ID (UID):** Numeric identifier assigned by the system.
- **Password:** Secret key used for authentication.
- **Group ID (GID):** Identifier for the group the user belongs to.
- **Home Directory:** Personal workspace assigned to the user.
- **Default Shell:** The command-line interface provided.

In Linux, account info is stored in:

- `/etc/passwd` (basic details)
- `/etc/shadow` (hashed passwords)
- `/etc/group` (group memberships)

### 3.2. Permissions

Permissions specify the access rights a user or group has over system objects such as files, directories, and devices.

## Common Permission Types:

- **Read (r)**: Permission to view contents.
- **Write (w)**: Permission to modify contents.
- **Execute (x)**: Permission to run a file or enter a directory.

### ♦ Linux File Permission Model:

Permissions are assigned to three categories:

- **Owner** (user who owns the file)
- **Group** (users in the file's group)
- **Others** (everyone else)

Example permission string:

`-rwxr-xr--`

- Owner: read, write, execute
- Group: read, execute
- Others: read only

## 3.3. Managing User Accounts

### Creation

- Linux:  

```
bash
sudo adduser username
```
- Windows: Via Control Panel or command line.

### Modification

- Change password:  

```
bash
sudo passwd username
```

- Change user shell or home directory:

```
bash
usermod -s /bin/bash username
```

## **Deletion**

- Linux:

```
bash
sudo deluser username
```

## **3. 4. Managing Permissions**

### **Commands in Linux:**

- Change file permissions:

```
bash
chmod [options] filename
```

- Change file ownership:

```
bash
chown user:group filename
```

## **3.5. Groups and Permission Simplification**

Groups collect users with similar permissions, making management easier.

- Create group:

```
bash
sudo groupadd developers
```

- Add user to group:

```
bash
sudo usermod -aG developers username
```

Files and directories can then be assigned permissions at the group level.

### **Review Questions**

1. What are the three basic permission types in Linux?
2. How do groups simplify permission management?

## 4.1 Environment Configuration

Environment configuration refers to the setup and customization of the operating system and software environment in which users and applications operate. It involves setting variables, paths, system parameters, and preferences that influence how programs run and interact with the system.

### Key Components of Environment Configuration

#### a. Environment Variables

- These are dynamic values that affect the processes and applications on a computer.
- Common environment variables include:
  - PATH — directories where executable programs are located.
  - HOME — the home directory of the current user.
  - USER — the username of the current user.
  - SHELL — the default shell used.
- Users and programs can read or modify these variables.

#### b. Configuration Files

- Files that store environment settings and preferences.
- Common configuration files (especially in Linux/Unix) include:
  - .bashrc, .bash\_profile — shell-specific configuration files for Bash.
  - /etc/profile — system-wide environment and startup programs.
  - /etc/environment — system-wide environment variables.

#### c. System Paths

- The PATH environment variable is critical. It lists directories that the shell searches for executable files.
- Modifying PATH allows users to run custom scripts or programs from any location.

#### d. Shell Configuration

- Users can customize their shell experience by editing shell configuration files to:
  - Set aliases for commands.
  - Customize prompts.

- Export environment variables.
- Define functions.

### Why is Environment Configuration Important?

- Ensures software and scripts run correctly by pointing to the right executables and libraries.
- Provides user-specific or system-wide customization.
- Helps automate tasks and improve productivity.
- Controls security and resource access (e.g., restricting paths).

### Review Questions

1. Why is environment configuration important for system administrators?

## 4.2 Administrative Tools in System Administration

### a. Shell Scripting

- A shell script is a text file containing a sequence of commands for the shell (command interpreter) to execute.
- Used to automate repetitive tasks, system management, and batch processing.

### Key Features:

- Can include loops, conditionals, and functions.
- Helps automate user and system administration tasks.
- Common shells: Bash, sh, ksh.

### Example:

```
bash
#!/bin/bash
echo "Backup started"
tar -czf /backup/home_$(date +%F).tar.gz /home/user
```

echo "Backup completed"

## b. AWK- Imp

- **AWK** is a powerful **text-processing language** and command-line tool used for **pattern scanning and processing**.
- Named after its creators: **Aho (American), Weinberger(American), and Kernighan(Canadian)**.
- Primarily used to extract and manipulate data from text files, especially structured text like logs, CSV( Comma Separated Values) files, or system reports.
- AWK reads input line by line, splits it into fields (columns), and applies actions based on patterns.

### Basic Structure of AWK Command

bash

```
awk 'pattern { action }' filename
```

- **pattern:** Condition to match (optional).
- **action:** What to do when pattern matches (optional).
- If no pattern is provided, action is applied to all lines.
- Fields in each line are accessed using \$1, \$2, ..., \$n.
- \$0 represents the entire line.

### Real-time Examples of AWK Usage

#### 1. Print Specific Columns from a File

Suppose you have a file students.txt:

```
101 John 85
102 Alice 90
103 Bob 78
```

To print only student names and marks:

bash

```
awk '{ print $2, $3 }' students.txt
```

### **Output:**

John 85  
Alice 90  
Bob 78

## **2. Filter Lines Based on Condition**

Print students who scored more than 80:

```
bash  
awk '$3 > 80 { print $2, $3 }' students.txt
```

### **Output:**

John 85  
Alice 90

## **Summary**

AWK is a versatile tool used for quick data extraction, text filtering, and report generation, making it invaluable for **system administrators** dealing with logs, reports, and configuration files.

## **c. Make**

- A build automation tool primarily used to compile and build software from source code.
- Uses a file called Makefile that defines rules and dependencies.

### **Key Features:**

- Automates compilation processes.
- Only recompiles files that have changed.
- Widely used in software development.

### Example Makefile snippet:

```
makefile
CopyEdit
all: program

program: main.o utils.o
    gcc -o program main.o utils.o

main.o: main.c
    gcc -c main.c

utils.o: utils.c
    gcc -c utils.c
```

### d. CRON Jobs

- CRON is a time-based job scheduler in Unix/Linux.
- Used to schedule repetitive tasks (jobs) to run automatically at specified times.

### Key Components:

- **crontab**: The file where scheduled jobs are defined.
- Each line specifies when and what to run.

### Crontab format:

```
* * * * * command_to_execute
- - - - -
| | | |
| | | | +----- Day of week (0-6, Sunday=0)
| | | +----- Month (1-12)
| | +----- Day of month (1-31)
| +----- Hour (0-23)
+----- Minute (0-59)
```

### Example:

Run backup script every day at 2 AM:

```
cron  
0 2 * * * /home/user/backup.sh
```

### Summary Table

| Tool            | Purpose                              | Common Usage                            |
|-----------------|--------------------------------------|-----------------------------------------|
| Shell Scripting | Automate command execution           | Backups, system monitoring              |
| AWK             | Text processing and pattern matching | Log analysis, report generation         |
| Make            | Build automation for software        | Compile source code efficiently         |
| CRON Jobs       | Schedule repetitive tasks            | Automated backups, updates, maintenance |

### Possible Questions

1. What are the benefits of using shell scripting in system administration?
2. How does AWK differ from other text processing tools?
3. Why is Make useful in software development?
4. What is the purpose of a crontab file?